



المعهد الوطني للبريد والهواصالات

المعهد الوطني للبريد والهواصالات

Institut National des Postes et Télécommunications

## INSITUT NATIONAL DES POSTES ET TELECOMMUNICATIONS

ARCHITECTURE ORIENTÉE SERVICES (SOA) ET  
DÉVELOPPEMENT WEB  
RAPPORT

---

### Mini-Projet – MyHeart : Système de Gestion des Soins de Santé

---

*Realisé par :*  
NAJMI YASSIN

*Encadrant :*  
M. Hassan FARSI

# Table des matières

|          |                                                                               |           |
|----------|-------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction et objectifs</b>                                              | <b>2</b>  |
| <b>2</b> | <b>Partie I – Conception et Architecture</b>                                  | <b>2</b>  |
| 2.1      | Identification des microservices . . . . .                                    | 2         |
| 2.2      | Diagramme d’architecture . . . . .                                            | 3         |
| 2.3      | Communication et bases de données . . . . .                                   | 3         |
| <b>3</b> | <b>Partie II – Implémentation</b>                                             | <b>3</b>  |
| 3.1      | Auth Service – Connexion et génération JWT . . . . .                          | 3         |
| 3.2      | Communication inter-services : Appointment → Billing → Notification . . . . . | 4         |
| 3.3      | Laboratory Service – FastAPI + MongoDB . . . . .                              | 4         |
| 3.4      | Frontend React.js . . . . .                                                   | 5         |
| <b>4</b> | <b>Partie III – Déploiement avec Docker</b>                                   | <b>6</b>  |
| 4.1      | Dockerfiles et Docker Compose . . . . .                                       | 6         |
| <b>5</b> | <b>Partie IV – Tests avec Postman</b>                                         | <b>7</b>  |
| 5.1      | Collection Postman – Structure . . . . .                                      | 7         |
| 5.2      | Scénario de test complet (bout en bout) . . . . .                             | 7         |
| 5.3      | Tests de sécurité et cas d’erreur . . . . .                                   | 8         |
| 5.4      | Health checks – Tous les services . . . . .                                   | 9         |
| <b>6</b> | <b>Dépôt GitHub</b>                                                           | <b>9</b>  |
| <b>7</b> | <b>Conclusion</b>                                                             | <b>10</b> |

# 1 Introduction et objectifs

MyHeart est un système hospitalier basé sur une architecture microservices. Chaque fonctionnalité métier est isolée dans un service indépendant, déployé dans son propre conteneur Docker et accessible via un API Gateway centralisé.

**Objectifs :** concevoir et implémenter 8 microservices communicants, développer un frontend React.js, et déployer l'ensemble avec Docker Compose.

GitHub : <https://github.com/YNJ05/MyHeart-Project>

## 2 Partie I – Conception et Architecture

### 2.1 Identification des microservices

| gray !20 # | Service              | Port | Responsabilité                             |
|------------|----------------------|------|--------------------------------------------|
| 0          | API Gateway          | 8080 | Point d'entrée unique, routage, JWT        |
| 1          | Auth Service         | 3000 | Inscription, connexion, tokens JWT         |
| 2          | Patient Service      | 3001 | CRUD dossiers patients                     |
| 3          | Appointment Service  | 3002 | Rendez-vous + communication inter-services |
| 4          | Billing Service      | 3003 | Facturation automatique, paiements         |
| 5          | Laboratory Service   | 3004 | Rapports d'analyses (FastAPI + MongoDB)    |
| 6          | Medical Records      | 3005 | Dossiers médicaux, diagnostics             |
| 7          | Pharmacy Service     | 3006 | Prescriptions, inventaire médicaments      |
| 8          | Notification Service | 3007 | Alertes et confirmations patients          |

TABLE 1 – Microservices du système MyHeart

**Justifications clés :** Billing est indépendant car il peut exister sans rendez-vous. Laboratory utilise MongoDB car les résultats d'analyses ont un schéma JSON variable. Les notifications sont isolées car transversales à plusieurs services.

## 2.2 Diagramme d'architecture

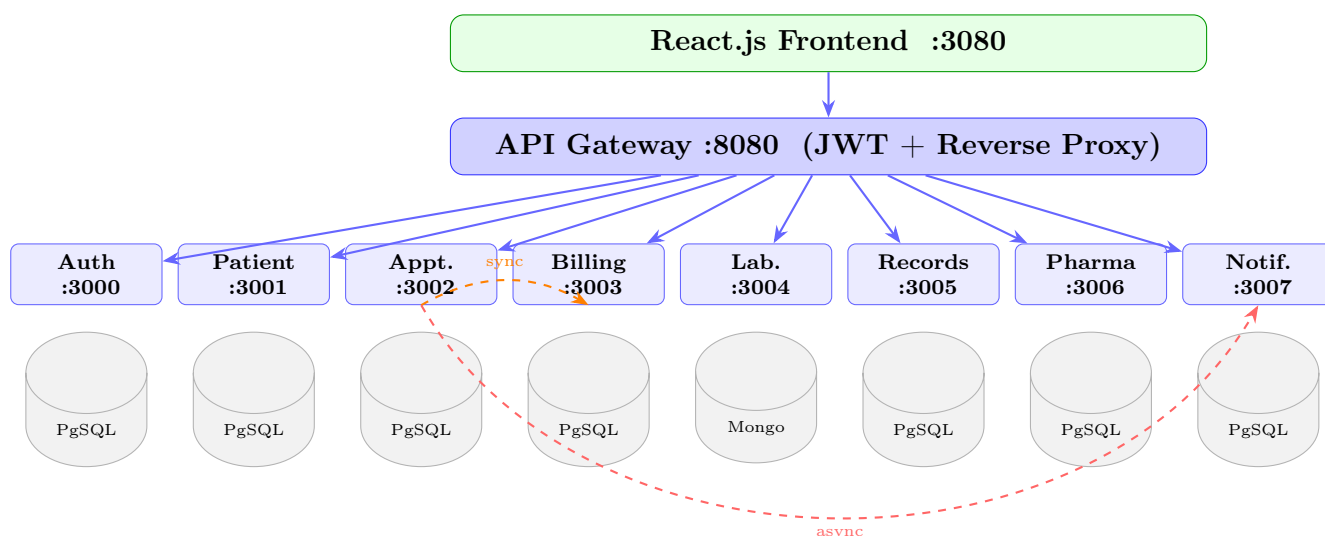


FIGURE 1 – Architecture complète MyHeart – flèches orange = communication inter-services

## 2.3 Communication et bases de données

| Source      | Destination  | Type                  | Raison                                        |
|-------------|--------------|-----------------------|-----------------------------------------------|
| Appointment | Billing      | REST Synchrones       | Facturation immédiate attendue par le patient |
| Appointment | Notification | Async (fire & forget) | Léger délai acceptable, non bloquant          |
| API Gateway | Auth Service | REST Synchrones       | Validation JWT sur chaque requête protégée    |

TABLE 2 – Communication inter-services — BDD : PostgreSQL (ACID) pour 7 services ; MongoDB (JSON flexible) pour le Laboratoire

## 3 Partie II – Implémentation

### 3.1 Auth Service – Connexion et génération JWT

Listing 1 – Auth Service – POST /login : vérification bcrypt + émission JWT

```
app.post('/login', async (req, res) => {
  const result = await pool.query('SELECT * FROM users WHERE email=$1', [req.body.email]);
  if (result.rowCount === 0) return res.status(401).json({ error: 'Identifiants invalides' });
  const valid = await bcrypt.compare(req.body.password, result.rows[0].password);
  if (!valid) return res.status(401).json({ error: 'Identifiants invalides' });
  const token = jwt.sign(
    { userId: result.rows[0].id, role: result.rows[0].role },
    JWT_SECRET, { expiresIn: '24h' }
  );
  res.json({ token, user: result.rows[0] });
});
```

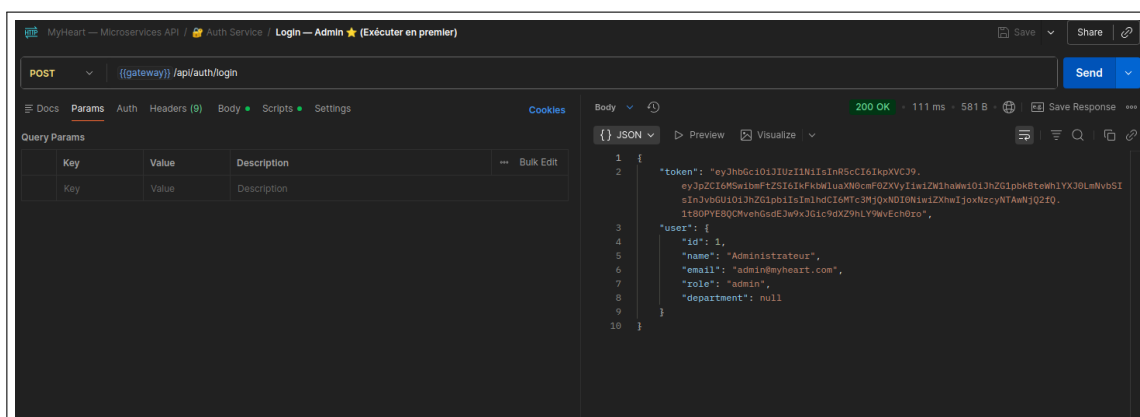


FIGURE 2 – POST /api/auth/login – Réponse avec token JWT (admin@myheart.com / Admin@1234)

### 3.2 Communication inter-services : Appointment → Billing → Notification

Listing 2 – Appointment Service – RDV avec facturation synchrone et notification asynchrone

```

app.post('/appointments', async (req, res) => {
  const appt = await createAppointment(req.body);

  // SYNCHRONE : le patient attend une confirmation de facturation
  await fetch(`${BILLING_URL}/billing`, {
    method: 'POST',
    body: JSON.stringify({ patient_id: appt.patient_id,
      description: 'Consultation - ${appt.specialty}', amount: 150.00 }),
  });

  // ASYNCHRONE (fire & forget) : notification sans bloquer la reponse
  setImmediate(() => fetch(`${NOTIFICATION_URL}/notifications`, {
    method: 'POST',
    body: JSON.stringify({ patient_id: appt.patient_id,
      title: 'RDV confirme', message: 'Votre RDV du ${appt.appointment_date}.' }),
  }));

  res.status(201).json(appt);
});

```

### 3.3 Laboratory Service – FastAPI + MongoDB

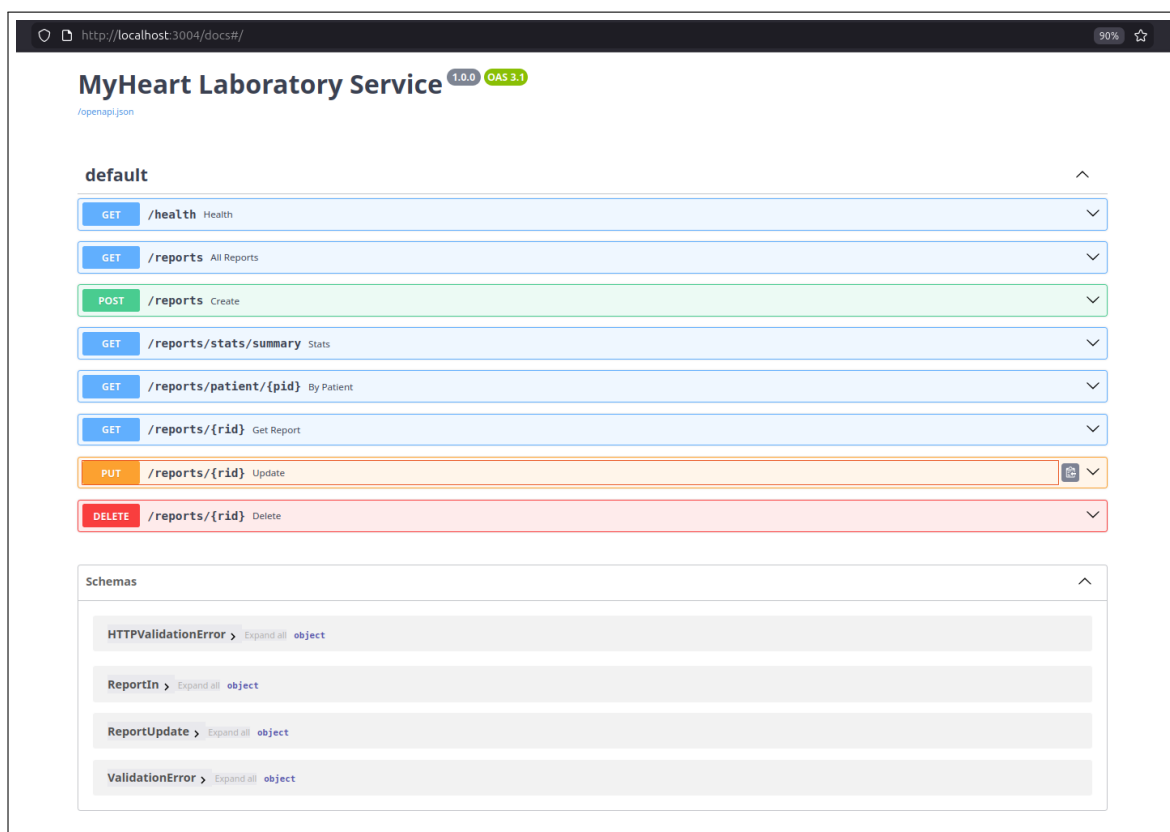
Listing 3 – Laboratory Service – Modèle Pydantic et endpoint POST /reports

```

class LabReport(BaseModel):
    patient_id: int
    test_type: str # Ex: "Hematologie"
    test_name: str # Ex: "NFS complete"
    results: dict # JSON flexible : structure variable selon le test
    status: str = "pending"

@app.post("/reports", status_code=201)
async def create_report(report: LabReport):
    doc = {**report.dict(), "created_at": datetime.utcnow().isoformat()}
    result = await db.reports.insert_one(doc)
    return serialize(await db.reports.find_one({"_id": result.inserted_id}))

```

FIGURE 3 – Swagger UI auto-généré par FastAPI (`http://localhost:3004/docs`)

### 3.4 Frontend React.js

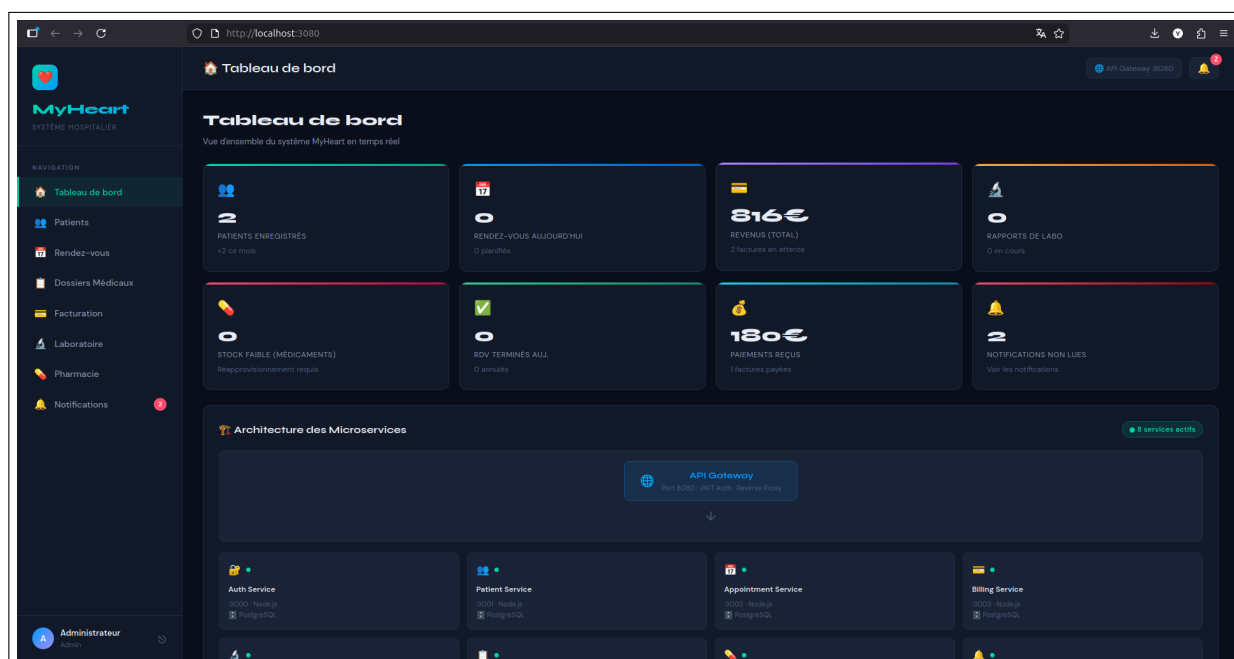


FIGURE 4 – Tableau de bord – Statistiques en temps réel et rendez-vous récents

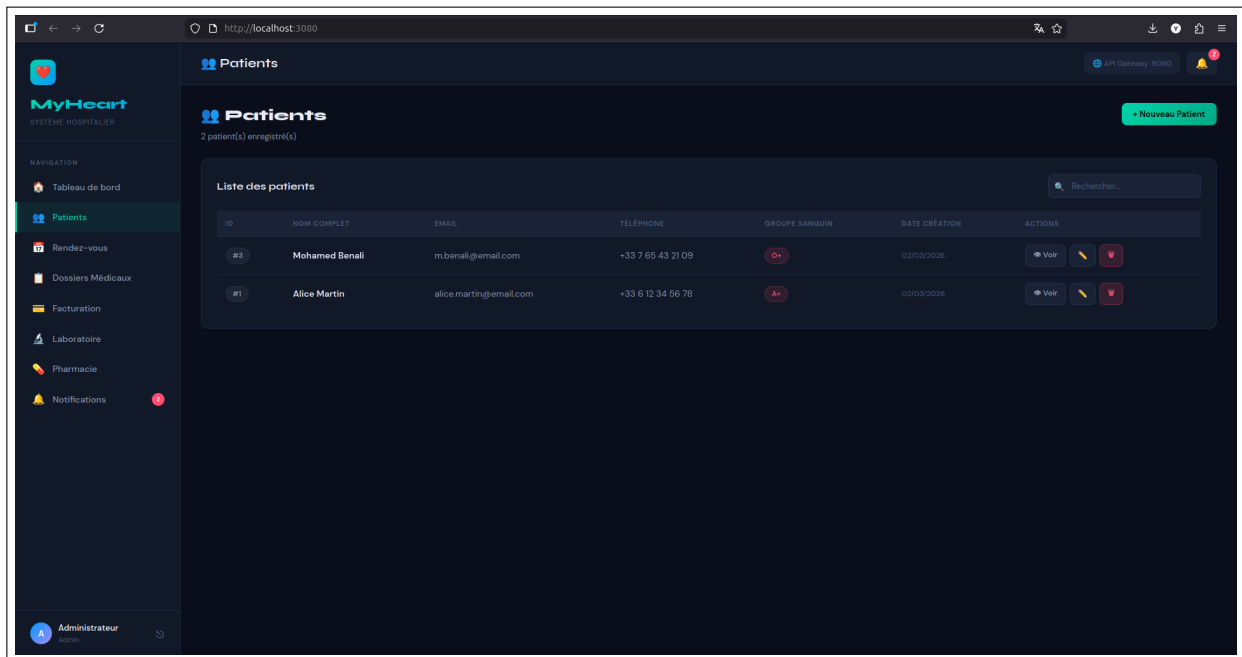


FIGURE 5 – Gestion des patients – Liste, recherche et modal de création/modification

## 4 Partie III – Déploiement avec Docker

### 4.1 Dockerfiles et Docker Compose

Listing 4 – Dockerfile Node.js (à gauche) et Frontend multi-stage (à droite)

```
# --- Services Node.js ---
FROM node:18-alpine
WORKDIR /app
COPY package.json .
RUN npm install --production
COPY src/ ./src/
EXPOSE 3001
CMD ["node", "src/index.js"]

# --- Frontend React + Nginx ---
FROM node:18-alpine AS build
WORKDIR /app
COPY package.json .
RUN npm install && npm run build
FROM nginx:alpine
COPY --from=build /app/build \
  /usr/share/nginx/html
EXPOSE 80
```

Listing 5 – docker-compose.yml – Healthcheck et depends\_on (extrait)

```
appointment-service:
  build: ./appointment-service
  ports: ["3002:3002"]
  environment:
    BILLING_SERVICE_URL: http://billing-service:3003
    NOTIFICATION_SERVICE_URL: http://notification-service:3007
  depends_on:
    postgres-appointment:
      condition: service_healthy      # Attend que la BDD soit prête

postgres-appointment:
  image: postgres:15-alpine
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 10s
    retries: 5
```

```

ynj-user@HUAMIE-015:~$ docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED        STATUS        PORTS                               NAMES
32ef0b9f7c49   myheart-frontend       "/docker-entrypoint..." 37 minutes ago Up 37 minutes 0.0.0.0:3080->80/tcp, [::]:3080->80/tcp   frontend
5d194a6abe4b   myheart-api-gateway     "/docker-entrypoint..." 37 minutes ago Up 37 minutes 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   api-gateway
b874d59d5721   myheart-laboratory-service "python maIn.py"         37 minutes ago Up 37 minutes 0.0.0.0:3004->3004/tcp, [::]:3004->3004/tcp   laboratory-service
3e2e35e94151   myheart-appointment-service "docker-entrypoint..." 37 minutes ago Up 37 minutes 0.0.0.0:3002->3002/tcp, [::]:3002->3002/tcp   appointment-service
b0cdd99622b5   myheart-auth-service    "docker-entrypoint..." 37 minutes ago Up 37 minutes 0.0.0.0:3008->3008/tcp, [::]:3008->3008/tcp   auth-service
8744182124bb   myheart-pharmacy-service "docker-entrypoint..." 37 minutes ago Up 37 minutes 0.0.0.0:3006->3006/tcp, [::]:3006->3006/tcp   pharmacy-service
914e590671c1   myheart-billing-service  "docker-entrypoint..." 37 minutes ago Up 37 minutes 0.0.0.0:3003->3003/tcp, [::]:3003->3003/tcp   billing-service
3e666c346cd4   myheart-notification-service "docker-entrypoint..." 37 minutes ago Up 37 minutes 0.0.0.0:3007->3007/tcp, [::]:3007->3007/tcp   notification-service
fef8f8361294   myheart-medical-records-service "docker-entrypoint..." 37 minutes ago Up 37 minutes 0.0.0.0:3005->3005/tcp, [::]:3005->3005/tcp   medical-records-service
59b887a93976   myheart-patient-service  "docker-entrypoint..." 37 minutes ago Up 37 minutes 0.0.0.0:3001->3001/tcp, [::]:3001->3001/tcp   patient-service

```

FIGURE 6 – docker ps – 10 conteneurs actifs (8 services + API Gateway + Frontend)

## 5 Partie IV – Tests avec Postman

### 5.1 Collection Postman – Structure

La collection **MyHeart — Microservices API** regroupe **63 requêtes** organisées en 10 dossiers couvrant l'ensemble des services. La variable `{{gateway}}` pointe vers `http://localhost:8080`.

| gray !20 Dossier     | Requêtes  | Couverture                                                       |
|----------------------|-----------|------------------------------------------------------------------|
| API Gateway          | 4         | Health, routes inconnues, accès sans/mauvais token (401/403)     |
| Auth Service         | 6         | Register, Login, Login erroné, profil, liste utilisateurs        |
| Patient Service      | 9         | CRUD complet, recherche, statistiques, 404                       |
| Appointment Service  | 9         | Création RDV ( déclenche Billing + Notification), filtres, stats |
| Billing Service      | 9         | Stats financières, liste, paiement carte/cash, création manuelle |
| Laboratory Service   | 11        | Bilan sanguin, glycémie (urgent), IRM, filtres, stats, update    |
| Medical Records      | 7         | Création, dossiers urgences, liste, dossiers par patient         |
| Pharmacy Service     | 9         | Prescriptions, délivrance, inventaire, stock faible, réappro     |
| Notification Service | 7         | Notifications, non-lues, compteur, marquer lu / tout lu          |
| Scénarios complets   | 6         | Workflow de bout en bout (patient → RDV → facture → paiement)    |
| <b>Total</b>         | <b>63</b> |                                                                  |

TABLE 3 – Structure de la collection Postman MyHeart — Microservices API

### 5.2 Scénario de test complet (bout en bout)

Le dossier **Scénarios de test complets** valide la chaîne entière du système :

Listing 6 – Séquence des 6 requêtes du scénario de bout en bout

```

SCENARIO 1      POST /api/patients
  Body: {"first_name":"Test","last_name":"Postman","blood_type":"O+"}
  -> Cr e le patient, sauvegarde {{patient_id}}

SCENARIO 2      POST /api/appointments
  Body: {"patient_id":{{patient_id}}, "specialty":"Cardiologie",
        "appointment_date":"2025-03-15T10:00:00"}
  -> Cr e le RDV ET d clenche automatiquement :
    [sync] POST /billing -> facture de 150.00 MAD (status: pending)
    [async] POST /notifications -> confirmation envoy e au patient

SCENARIO 3      GET /api/billing/patient/{{patient_id}}
  -> V rifie la facture auto-g n r e , sauvegarde {{invoice_id}}

SCENARIO 4      GET /api/notifications
  -> V rifie la notification de confirmation re ue

SCENARIO 5      PUT /api/billing/{{invoice_id}}/pay
  Body: {"payment_method":"card"}

```



```
-> Paie la facture (status: paid)

SCENARIO 6      POST /api/records
Body: {"patient_id":{{patient_id}}, "diagnosis":"Hypertension",
      "treatment":"Amlodipine 5mg"}
-> Complète le dossier médical du patient
```

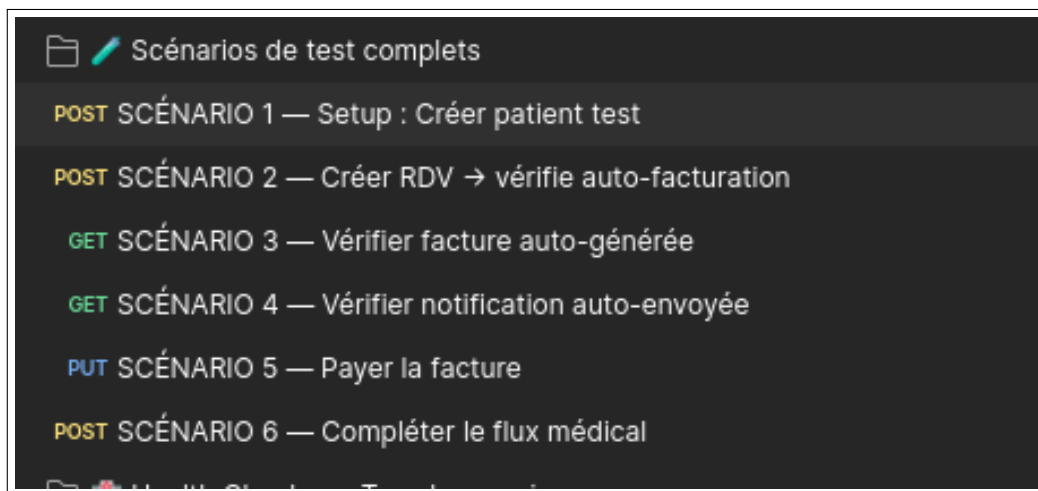


FIGURE 7 – Dossier « Scénarios de test complets » – Chaîne Appointment → Billing → Notification validée

### 5.3 Tests de sécurité et cas d'erreur

Le dossier **API Gateway** inclut des tests de sécurité explicites :

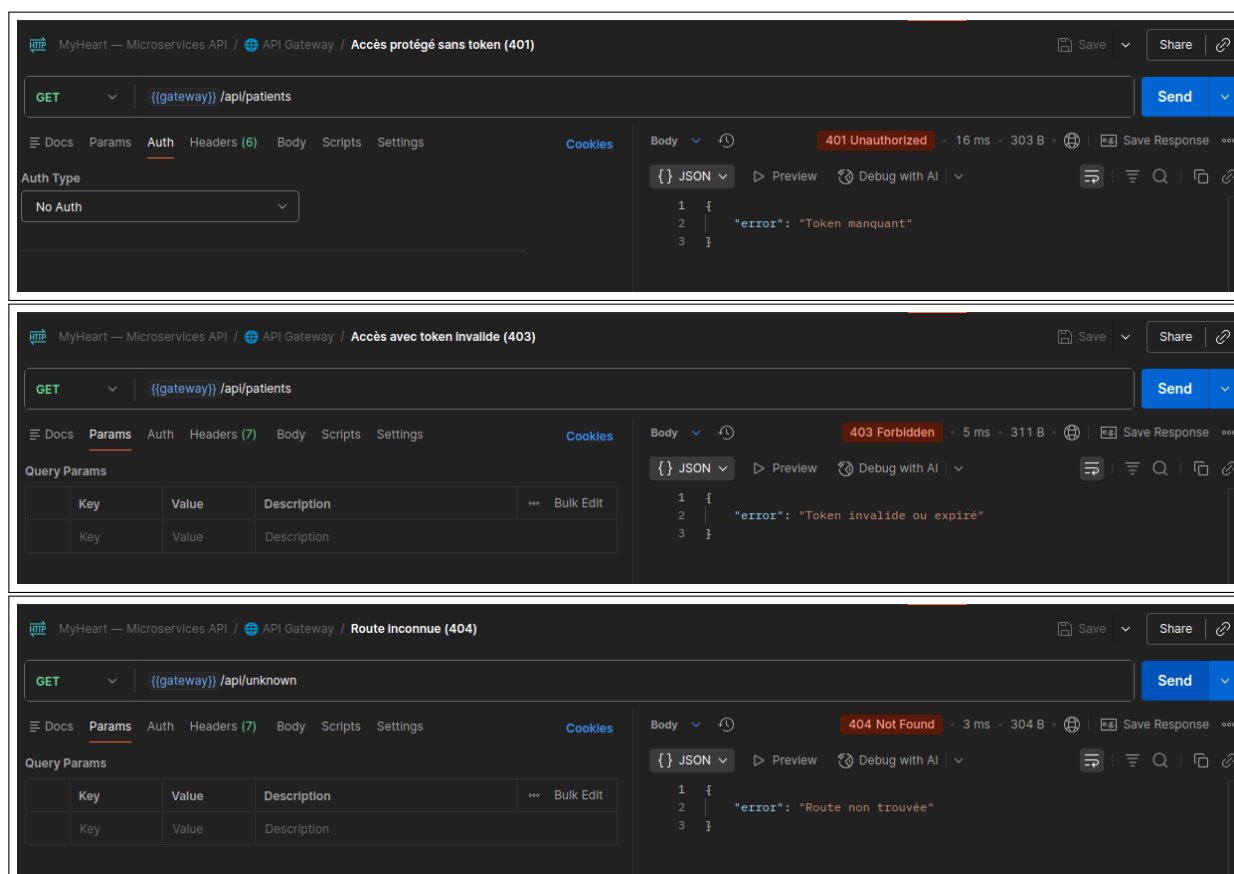


FIGURE 8 – Tests de sécurité : accès sans token (401) et avec token invalide (403)

- **GET /api/patients sans token** → 401 Unauthorized (Token manquant)
- **GET /api/patients avec token invalide** → 403 Forbidden (Token expiré ou invalide)
- **GET /api/unknown** → 404 Not Found (Route inexistante dans le Gateway)

## 5.4 Health checks – Tous les services



FIGURE 9 – Dossier « Health Checks » – Les 9 services répondent {"status":"OK"}

## 6 Dépôt GitHub

Dépôt GitHub : <https://github.com/YNJ05/MyHeart-Project>

## 7 Conclusion

Ce mini-projet nous a permis de parcourir l'intégralité du cycle de développement d'une application microservices, de la conception architecturale jusqu'au déploiement conteneurisé.

À travers la réalisation du système **MyHeart**, nous avons mis en pratique les principes fondamentaux du paradigme SOA appliqués aux microservices.

Sur le plan de la **conception**, l'analyse des besoins métier hospitaliers nous a conduits à identifier 8 services autonomes, chacun responsable d'un domaine fonctionnel unique (Single Responsibility Principle). Le choix du *Database per Service pattern* garantit l'isolation totale des données et permet une évolution indépendante de chaque service. L'utilisation de **PostgreSQL** pour les données structurées et de **MongoDB** pour les rapports de laboratoire illustre concrètement l'adaptation du SGBD aux besoins spécifiques de chaque service.

Sur le plan de l'**implémentation**, la communication *synchrone* entre l'Appointment Service et le Billing Service garantit la cohérence immédiate de la facturation, tandis que la communication *asynchrone* (fire & forget) vers le Notification Service préserve les performances sans sacrifier la fonctionnalité. L'API Gateway centralise l'authentification JWT et le routage, évitant la duplication de la logique de sécurité dans chaque service.

La **validation** du système via une collection Postman de 63 requêtes, incluant des scénarios de bout en bout et des tests de sécurité (401, 403, 404), démontre la robustesse de l'ensemble. Le déploiement Docker avec healthchecks et dépendances conditionnelles (`service_healthy`) assure un démarrage fiable et ordonné de l'ensemble du cluster.

**Perspectives d'amélioration :** l'intégration d'un message broker tel que RabbitMQ ou Apache Kafka remplacerait avantageusement le fire & forget pour garantir la livraison des messages même en cas de panne. La mise en place d'un Circuit Breaker renforcerait la résilience face aux défaillances inter-services. Enfin, un déploiement sur **Kubernetes** permettrait la scalabilité horizontale automatique, et l'ajout de **Prometheus + Grafana** assurerait une supervision temps réel de l'ensemble des microservices.